МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»
Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

Отчет к лабораторной работе № 7

Тема: **Поведенческие паттерны: Команда, Итератор**

Дисциплина: **«Паттерны проектирования»**

Выполнила:

Студентка группы ПИЖ-б-о-23-2
направление подготовки 09.03.04
«Программная инженерия»

Силина Оксана Романовна

Проверил:

Ассистент ДЦРСИЭ

Белик А. И.

Ставрополь 2026

Цель работы: изучить принцип работы поведенческих паттернов проектирования «Команда» и «Итератор» и реализовать прикладную задачу расчёта траектории движения тела с использованием данных паттернов.

Задача: разработать приложение для расчёта параметров движения тела, брошенного под углом к горизонту, с использованием двух поведенческих паттернов проектирования.

Выбранные паттерны:

- Команда — для инкапсуляции запросов расчёта траектории в виде объектов, что позволяет хранить историю, повторять и отменять действия;
- Итератор — для последовательного обхода точек траектории без раскрытия внутреннего устройства коллекции.

Индивидуальный вариант: 3. Разработка приложения «Траектория движения»

Репозиторий: https://github.com/tsukuri1/patterns

1. Описание паттернов

1.1. Команда (Command)

Команда — это поведенческий паттерн проектирования, который превращает запрос в объект. Это позволяет параметризовать клиенты командами, ставить команды в очередь, логировать их и поддерживать отмену операций. Каждая команда инкапсулирует получателя и параметры вызова, реализуя общий интерфейс с методом execute().

Благодаря паттерну «Команда» каждый расчёт оформляется в виде объекта CalculateTrajectoryCommand, который сохраняется в CommandHistory. Это даёт пользователю три возможности: посмотреть историю выполненных расчётов с их параметрами, отменить последнюю команду (её результат сбрасывается), а также отделить инициатора запроса (интерфейс) от получателя (TrajectoryService), что упрощает расширение приложения.

Применение в задаче:

Интерфейс Command задаёт метод execute() и undo(). Класс CalculateTrajectoryCommand инкапсулирует запрос на расчёт с параметрами v0, angle, steps и получателем — сервисом TrajectoryService. Класс CommandHistory хранит выполненные команды, позволяя повторить расчёт и откатить его. Клиент (интерфейс приложения) работает с командами через историю, не зная деталей их выполнения.

1.2. Итератор (Iterator)

Итератор — это поведенческий паттерн проектирования, который предоставляет способ последовательного доступа к элементам составного объекта, не раскрывая его внутреннего представления. Итератор инкапсулирует обход коллекции и хранит текущую позицию.

Благодаря паттерну «Итератор» пользователь может последовательно обойти все точки траектории через TrajectoryIterator, не имея доступа к внутреннему списку TrajectoryCollection. Кнопка «Обойти точки (итератор)» в интерфейсе демонстрирует именно такой обход: точки выводятся по одной через has_next() и next().

Применение в задаче:

Коллекция TrajectoryCollection хранит точки траектории. Итератор TrajectoryIterator обеспечивает последовательный обход точек, скрывая внутренний список от клиента. Клиентский код получает точки через методы next() и has_next() и не имеет доступа к внутренней структуре коллекции.

2. Структура приложения

Приложение состоит из следующих компонентов:

Компонент	Назначение
Command	Общий интерфейс команды
CalculateTrajectoryCommand	Команда расчёта траектории
CommandHistory	История выполненных команд

TrajectoryCollection	Коллекция точек траектории (Итератор)
TrajectoryIterator	Итератор для обхода точек
TrajectoryApp	Графический интерфейс приложения

3. Диаграмма классов

Для отображения структуры разработанного приложения была создана UML-диаграмма классов.

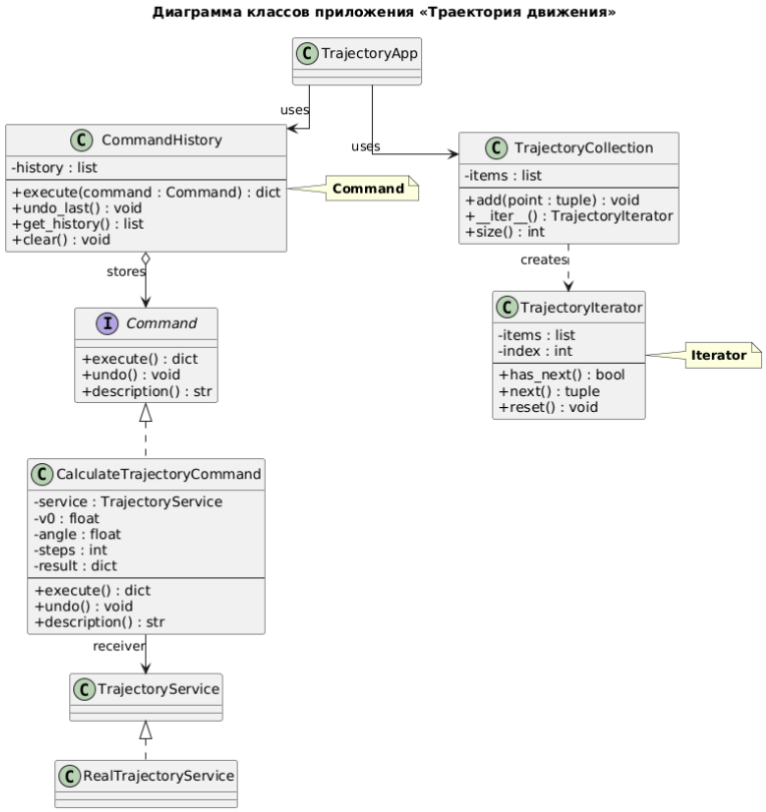


Рисунок 1 — Диаграмма классов приложения «Траектория движения»

Диаграмма содержит:

- Интерфейс Command, класс CalculateTrajectoryCommand и историю CommandHistory (паттерн «Команда»);
- Коллекцию TrajectoryCollection и итератор TrajectoryIterator (паттерн «Итератор»);
- Класс TrajectoryApp — интерфейс приложения.

4. Реализация паттерна «Заместитель»

4.1. Паттерн «Команда»

Command — общий интерфейс. CalculateTrajectoryCommand инкапсулирует параметры расчёта и получателя (TrajectoryService). Метод execute() выполняет расчёт и сохраняет результат, undo() сбрасывает сохранённый результат. CommandHistory хранит выполненные команды, позволяет выполнять новые и отменять последнюю.

```
import math


class TrajectoryService:
    """Получатель команды."""
    def calculate(self, v0, angle):
        raise NotImplementedError

    def points(self, v0, angle, steps):
        raise NotImplementedError


class RealTrajectoryService(TrajectoryService):
    def __init__(self, g=9.81):
        self.g = g

    def calculate(self, v0, angle):
        if v0 <= 0:
            raise ValueError("Скорость должна быть > 0")
        if not (0 < angle < 90):
            raise ValueError("Угол должен быть в диапазоне (0; 90)")
        a = math.radians(angle)
        T = 2 * v0 * math.sin(a) / self.g
        H = (v0 ** 2) * (math.sin(a) ** 2) / (2 * self.g)
        L = (v0 ** 2) * math.sin(2 * a) / self.g
        return {"time": T, "height": H, "range": L}

    def points(self, v0, angle, steps):
        res = self.calculate(v0, angle)
        T = res["time"]
        a = math.radians(angle)
        pts = []
```

```
            for i in range(steps + 1):
                t = T * i / steps
                x = v0 * math.cos(a) * t
                y = v0 * math.sin(a) * t - 0.5 * self.g * t ** 2
                pts.append((round(x, 3), round(y, 3)))
            return pts


    class Command:
        """Общий интерфейс команды."""
        def execute(self):
            raise NotImplementedError

        def undo(self):
            raise NotImplementedError

        def description(self):
            raise NotImplementedError


    class CalculateTrajectoryCommand(Command):
        """Команда расчёта траектории."""
        def __init__(self, service: TrajectoryService, v0, angle, steps):
            self.service = service
            self.v0 = v0
            self.angle = angle
            self.steps = steps
            self.result = None

        def execute(self):
            self.result = self.service.calculate(self.v0, self.angle)
            return self.result

        def undo(self):
            self.result = None

        def description(self):
            return f"Расчёт: v0={self.v0}, angle={self.angle},
steps={self.steps}"


    class CommandHistory:
        """История выполненных команд."""
        def __init__(self):
            self.history = []
```

```
    def execute(self, command: Command):
        result = command.execute()
        self.history.append(command)
        return result

    def undo_last(self):
        if self.history:
            command = self.history.pop()
            command.undo()
            return command
        return None

    def get_history(self):
        return [cmd.description() for cmd in self.history]

    def clear(self):
        self.history.clear()
```

4.2. Паттерн «Итератор»

TrajectoryCollection хранит список точек и умеет создавать итератор. TrajectoryIterator хранит ссылку на список и текущий индекс, реализуя методы has_next(), next() и reset(). Клиент обходит коллекцию, не зная, что внутри — список, кортеж или другая структура.

```
class TrajectoryIterator:
    """Итератор для обхода точек траектории."""
    def __init__(self, items):
        self.items = items
        self.index = 0

    def has_next(self):
        return self.index < len(self.items)

    def next(self):
        if not self.has_next():
            raise StopIteration("Точек больше нет")
        item = self.items[self.index]
        self.index += 1
        return item
```

```
    def reset(self):
        self.index = 0


class TrajectoryCollection:
    """Коллекция точек траектории."""
    def __init__(self):
        self.items = []

    def add(self, point):
        self.items.append(point)

    def __iter__(self):
        return TrajectoryIterator(self.items)

    def size(self):
        return len(self.items)
```

4.3 Графический интерфейс

Интерфейс построен так, чтобы обе возможности паттернов были видны пользователю. Кнопка «Выполнить расчёт (команда)» создаёт объект CalculateTrajectoryCommand и передаёт его в CommandHistory — так демонстрируется паттерн «Команда». Кнопки «Показать историю команд» и «Отменить последнюю команду» работают с той же историей: первая выводит список выполненных команд, вторая удаляет последнюю и сбрасывает её результат. Кнопка «Обойти точки (итератор)» создаёт TrajectoryIterator для TrajectoryCollection и последовательно выводит точки — так демонстрируется паттерн «Итератор».

```
import tkinter as tk
from tkinter import messagebox


class TrajectoryApp:
    def __init__(self, root):
        self.root = root
        root.title("Траектория движения")

        self.service = RealTrajectoryService()
```

```
        self.history = CommandHistory()
        self.collection = TrajectoryCollection()
        self.last_result = None

        tk.Label(root, text="Начальная скорость (м/с):").grid(row=0,
column=0, sticky="w")
        self.v0_entry = tk.Entry(root)
        self.v0_entry.insert(0, "20")
        self.v0_entry.grid(row=0, column=1)

        tk.Label(root, text="Угол броска (°):").grid(row=1, column=0,
sticky="w")
        self.angle_entry = tk.Entry(root)
        self.angle_entry.insert(0, "45")
        self.angle_entry.grid(row=1, column=1)

        tk.Label(root, text="Шагов траектории:").grid(row=2,
column=0, sticky="w")
        self.steps_entry = tk.Entry(root)
        self.steps_entry.insert(0, "10")
        self.steps_entry.grid(row=2, column=1)

        tk.Button(root, text="Выполнить расчёт (команда)",
                  command=self.execute_command).grid(row=3, column=0,
columnspan=2, pady=3)
        tk.Button(root, text="Отменить последнюю команду",
                  command=self.undo_command).grid(row=4, column=0,
columnspan=2)
        tk.Button(root, text="Показать историю команд",
                  command=self.show_history).grid(row=5, column=0,
columnspan=2)
        tk.Button(root, text="Обойти точки (итератор)",
                  command=self.iterate_points).grid(row=6, column=0,
columnspan=2)

        self.result = tk.Text(root, width=55, height=14)
        self.result.grid(row=7, column=0, columnspan=2)

    def _read_inputs(self):
        v0 = float(self.v0_entry.get())
        angle = float(self.angle_entry.get())
        steps = int(self.steps_entry.get())
        return v0, angle, steps

    def execute_command(self):
```

```
            try:
                v0, angle, steps = self._read_inputs()
                command = CalculateTrajectoryCommand(self.service, v0,
angle, steps)
                res = self.history.execute(command)
                self.last_result = res

                # Заполняем коллекцию точек траектории
                self.collection = TrajectoryCollection()
                pts = self.service.points(v0, angle, steps)
                for p in pts:
                    self.collection.add(p)

                self.result.delete("1.0", tk.END)
                self.result.insert(tk.END,
                    f"Команда выполнена:\n{command.description()}\n\n"
                    f"Время полёта: {res['time']:.3f} с\n"
                    f"Макс. высота: {res['height']:.3f} м\n"
                    f"Дальность: {res['range']:.3f} м\n"
                    f"Точек в коллекции: {self.collection.size()}\n")
                self.root.update_idletasks()
            except ValueError as e:
                messagebox.showerror("Ошибка ввода", str(e))
            except Exception as e:
                messagebox.showerror("Ошибка", str(e))

        def undo_command(self):
            cmd = self.history.undo_last()
            if cmd:
                self.last_result = None
                messagebox.showinfo("Отмена",
                                    f"Отменена команда:\n{cmd.description()}")
            else:
                messagebox.showinfo("Отмена", "История команд пуста.")

        def show_history(self):
            log = self.history.get_history()
            text = "\n".join(log) if log else "(история пуста)"
            messagebox.showinfo("История команд", text)

        def iterate_points(self):
            """Демонстрация паттерна Итератор."""
            if self.collection.size() == 0:
                messagebox.showinfo("Итератор", "Коллекция пуста.
Сначала выполните расчёт.")
```

```
            return
        iterator = self.collection.__iter__()
        self.result.delete("1.0", tk.END)
        self.result.insert(tk.END, "Обход точек через итератор:\n")
        i = 0
        while iterator.has_next():
            x, y = iterator.next()
            self.result.insert(tk.END, f"  {i}: x={x}, y={y}\n")
            i += 1
        self.result.insert(tk.END, f"\nВсего пройдено точек: {i}\n")


if __name__ == "__main__":
    root = tk.Tk()
    app = TrajectoryApp(root)
    root.mainloop()
```

5. Результаты работы программы

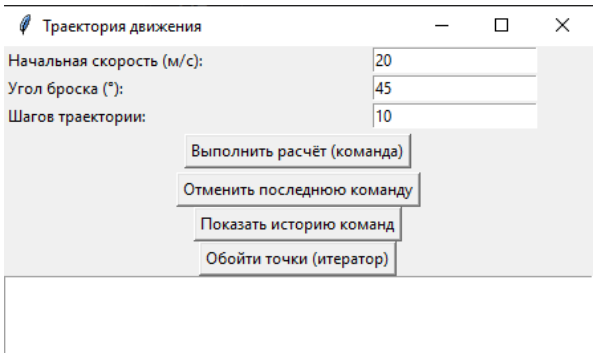


Рисунок 2 — Главное окно приложения

Пользователь вводит скорость, угол и число шагов. Каждый расчёт оформляется как команда и сохраняется в истории.

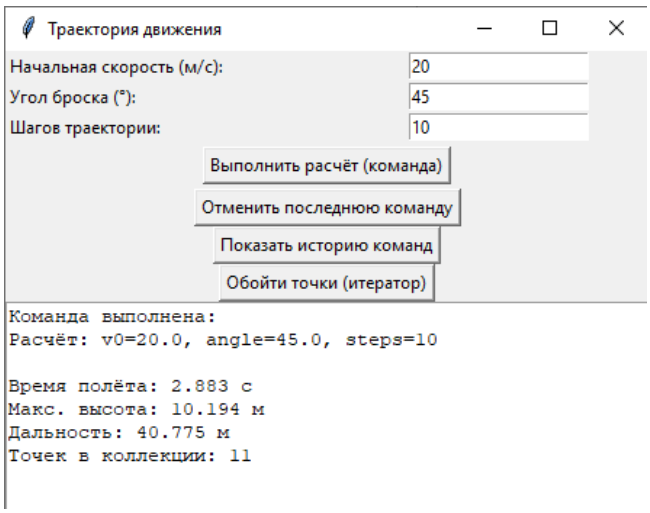


Рисунок 3 — Выполнение команды расчёта

При v0 = 20 м/с, угол = 45°, шагов = 10:

- Время полёта: 2.883 с
- Макс. высота: 10.194 м
- Дальность: 40.774 м
- Точек в коллекции: 11

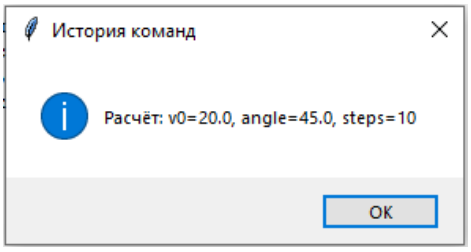


Рисунок 4 — История команд

После нескольких расчётов история содержит описания выполненных команд — видно, с какими параметрами они выполнялись.

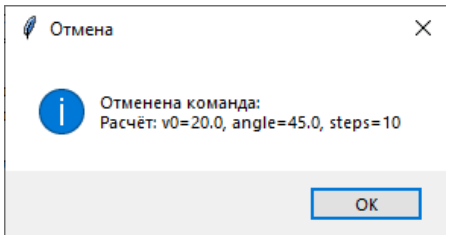


Рисунок 5 — Отмена последней команды

При нажатии «Отменить последнюю команду» последняя команда удаляется из истории, её результат сбрасывается. Клиентский код не знает деталей выполнения — он просто работает с историей команд.

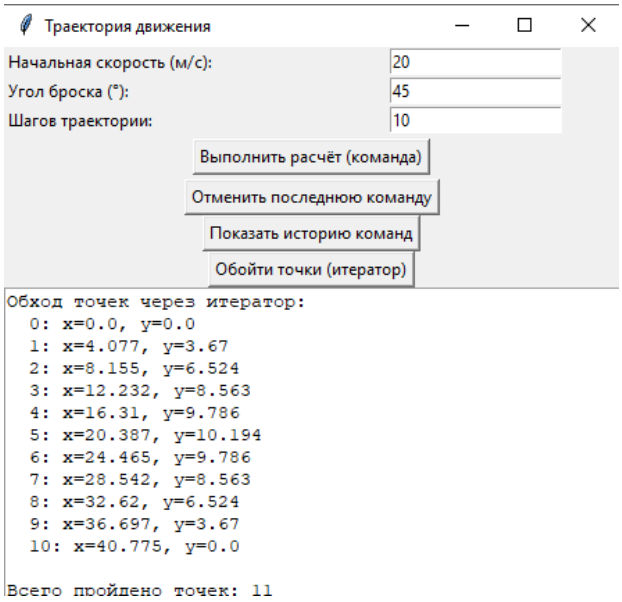


Рисунок 6 — Обход точек через итератор

Все точки траектории выводятся последовательно через TrajectoryIterator: видно, что клиент получает точки по одной, не имея доступа к внутреннему списку коллекции.

6. Исходный код

```
import math
import tkinter as tk
from tkinter import messagebox


class TrajectoryService:
    """Получатель команды."""
    def calculate(self, v0, angle):
        raise NotImplementedError

    def points(self, v0, angle, steps):
        raise NotImplementedError
```

```
class RealTrajectoryService(TrajectoryService):
    def __init__(self, g=9.81):
        self.g = g

    def calculate(self, v0, angle):
        if v0 <= 0:
            raise ValueError("Скорость должна быть > 0")
        if not (0 < angle < 90):
            raise ValueError("Угол должен быть в диапазоне (0; 90)")
        a = math.radians(angle)
        T = 2 * v0 * math.sin(a) / self.g
        H = (v0 ** 2) * (math.sin(a) ** 2) / (2 * self.g)
        L = (v0 ** 2) * math.sin(2 * a) / self.g
        return {"time": T, "height": H, "range": L}

    def points(self, v0, angle, steps):
        res = self.calculate(v0, angle)
        T = res["time"]
        a = math.radians(angle)
        pts = []
        for i in range(steps + 1):
            t = T * i / steps
            x = v0 * math.cos(a) * t
            y = v0 * math.sin(a) * t - 0.5 * self.g * t ** 2
            pts.append((round(x, 3), round(y, 3)))
        return pts

class Command:
    """Общий интерфейс команды."""
    def execute(self):
        raise NotImplementedError

    def undo(self):
        raise NotImplementedError

    def description(self):
        raise NotImplementedError


class CalculateTrajectoryCommand(Command):
    """Команда расчёта траектории."""
    def __init__(self, service: TrajectoryService, v0, angle, steps):
        self.service = service
        self.v0 = v0
```

```
        self.angle = angle
        self.steps = steps
        self.result = None

    def execute(self):
        self.result = self.service.calculate(self.v0, self.angle)
        return self.result

    def undo(self):
        self.result = None

    def description(self):
        return f"Расчёт: v0={self.v0}, angle={self.angle}, steps={self.steps}"


class CommandHistory:
    """История выполненных команд."""
    def __init__(self):
        self.history = []

    def execute(self, command: Command):
        result = command.execute()
        self.history.append(command)
        return result

    def undo_last(self):
        if self.history:
            command = self.history.pop()
            command.undo()
            return command
        return None

    def get_history(self):
        return [cmd.description() for cmd in self.history]

    def clear(self):
        self.history.clear()


class TrajectoryIterator:
    """Итератор для обхода точек траектории."""
    def __init__(self, items):
        self.items = items
        self.index = 0
```

```
        def has_next(self):
            return self.index < len(self.items)

        def next(self):
            if not self.has_next():
                raise StopIteration("Точек больше нет")
            item = self.items[self.index]
            self.index += 1
            return item

        def reset(self):
            self.index = 0


    class TrajectoryCollection:
        """Коллекция точек траектории."""
        def __init__(self):
            self.items = []

        def add(self, point):
            self.items.append(point)

        def __iter__(self):
            return TrajectoryIterator(self.items)

        def size(self):
            return len(self.items)


    class TrajectoryApp:
        def __init__(self, root):
            self.root = root
            root.title("Траектория движения")

            self.service = RealTrajectoryService()
            self.history = CommandHistory()
            self.collection = TrajectoryCollection()
            self.last_result = None

            tk.Label(root, text="Начальная скорость (м/с):").grid(row=0,
column=0, sticky="w")
            self.v0_entry = tk.Entry(root)
            self.v0_entry.insert(0, "20")
            self.v0_entry.grid(row=0, column=1)
```

```
        tk.Label(root, text="Угол броска (°):").grid(row=1, column=0,
sticky="w")
        self.angle_entry = tk.Entry(root)
        self.angle_entry.insert(0, "45")
        self.angle_entry.grid(row=1, column=1)

        tk.Label(root, text="Шагов траектории:").grid(row=2,
column=0, sticky="w")
        self.steps_entry = tk.Entry(root)
        self.steps_entry.insert(0, "10")
        self.steps_entry.grid(row=2, column=1)

        tk.Button(root, text="Выполнить расчёт (команда)",
                  command=self.execute_command).grid(row=3,
column=0, columnspan=2, pady=3)
        tk.Button(root, text="Отменить последнюю команду",
                  command=self.undo_command).grid(row=4, column=0,
columnspan=2)
        tk.Button(root, text="Показать историю команд",
                  command=self.show_history).grid(row=5, column=0,
columnspan=2)
        tk.Button(root, text="Обойти точки (итератор)",
                  command=self.iterate_points).grid(row=6, column=0,
columnspan=2)

        self.result = tk.Text(root, width=55, height=14)
        self.result.grid(row=7, column=0, columnspan=2)

    def _read_inputs(self):
        v0 = float(self.v0_entry.get())
        angle = float(self.angle_entry.get())
        steps = int(self.steps_entry.get())
        return v0, angle, steps

    def execute_command(self):
        try:
            v0, angle, steps = self._read_inputs()
            command = CalculateTrajectoryCommand(self.service, v0,
angle, steps)
            res = self.history.execute(command)
            self.last_result = res

            self.collection = TrajectoryCollection()
            pts = self.service.points(v0, angle, steps)
```

```
                for p in pts:
                    self.collection.add(p)

                self.result.delete("1.0", tk.END)
                self.result.insert(tk.END,
                    f"Команда выполнена:\n{command.description()}\n\n"
                    f"Время полёта: {res['time']:.3f} с\n"
                    f"Макс. высота: {res['height']:.3f} м\n"
                    f"Дальность: {res['range']:.3f} м\n"
                    f"Точек в коллекции: {self.collection.size()}\n")
                self.root.update_idletasks()
            except ValueError as e:
                messagebox.showerror("Ошибка ввода", str(e))
            except Exception as e:
                messagebox.showerror("Ошибка", str(e))

        def undo_command(self):
            cmd = self.history.undo_last()
            if cmd:
                self.last_result = None
                messagebox.showinfo("Отмена",
                                    f"Отменена команда:\n{cmd.description()}")
            else:
                messagebox.showinfo("Отмена", "История команд пуста.")

        def show_history(self):
            log = self.history.get_history()
            text = "\n".join(log) if log else "(история пуста)"
            messagebox.showinfo("История команд", text)

        def iterate_points(self):
            if self.collection.size() == 0:
                messagebox.showinfo("Итератор",    "Коллекция    пуста.
Сначала выполните расчёт.")
                return
            iterator = self.collection.__iter__()
            self.result.delete("1.0", tk.END)
            self.result.insert(tk.END, "Обход точек через итератор:\n")
            i = 0
            while iterator.has_next():
                x, y = iterator.next()
                self.result.insert(tk.END, f"  {i}: x={x}, y={y}\n")
                i += 1
            self.result.insert(tk.END, f"\nВсего пройдено точек: {i}\n")
```

```
    if __name__ == "__main__":
        root = tk.Tk()
        app = TrajectoryApp(root)
root.mainloop()
```

Вывод:

В ходе выполнения лабораторной работы было разработано приложение «Траектория движения», реализующее расчёт параметров движения тела, брошенного под углом к горизонту.

В приложении применены два поведенческих паттерна проектирования:

- Команда — для инкапсуляции запросов расчёта траектории в виде объектов CalculateTrajectoryCommand. История CommandHistory хранит выполненные команды, позволяет отменять последнюю и просматривать журнал. Это отделяет инициатора запроса (интерфейс) от получателя (TrajectoryService) и открывает возможность повторного выполнения и отката.
- Итератор — для последовательного обхода точек траектории через TrajectoryIterator. Клиентский код получает точки по одной через has_next() и next() и не зависит от внутреннего устройства TrajectoryCollection.

Таким образом, была изучена практическая реализация двух поведенческих паттернов и получено работоспособное приложение с графическим интерфейсом для расчёта и визуализации траектории движения тела.